How a Crash Prediction is Made: Stage by Stage

The Input: 66 Features Per Road Segment

Before any prediction, the model receives a snapshot of a road segment at a specific time. This snapshot contains 66 variables — things like:

- Historical crash counts on that segment (1 day ago, 7 days ago, 30 days ago)
- Road type (arterial, highway, residential)
- Weather conditions
- Time of day, day of week, month
- Nearby intersection density

Every road segment × every 24-hour window gets its own row with these 66 values.

Stage 1: "Will a crash happen?"

Model type: Gradient Boosted Decision Trees (300 trees, depth 6)

Each tree is a series of yes/no questions about the features:

```
Is it freezing? → Yes
  Is it an arterial road? → Yes
    Did this segment have a crash in the last 7 days? → Yes
      → High risk score
```

Each tree is shallow (max 6 questions deep), but 300 of them are chained together — each one trained to correct the errors the previous trees made. This sequential correction is what "boosting" means.

The output is a **raw score** that represents how risky the model thinks this window is. Higher = more likely a crash.

What the sample weights do here: Zero-crash windows are weighted ~65× their actual count in training, so the model can't cheat by ignoring crashes. Every missed crash costs the model 65× as much in its error calculation.

Calibration: Making the Score an Honest Probability

The raw score from Stage 1 is good for *ranking* roads (this one is riskier than that one) but the absolute number isn't reliable as a probability.

Calibration fixes this using the **validation set** — data from 2019–2021 that the model never trained on:

- Run the model on the validation set, get raw scores
- Group windows by their score (e.g., all windows that scored 0.40–0.45)
- Measure what fraction of each group actually had a crash
- Learn a mapping: raw score → true observed rate

This mapping is monotone — it can only stretch or compress the scale, never reorder predictions. After calibration, a score of 0.05 means roughly 5% of historically similar windows had crashes.

Why this matters for routing: The routing engine uses P(crash) directly as a penalty on road edges. Uncalibrated probabilities would over- or under-penalize roads, sending drivers the wrong way.

Stage 2: "If a crash happens, how many?"

Model type: Gradient Boosted Regressor with Poisson loss

Trained only on: Windows where at least one crash occurred (~1.5% of data)

Stage 2 never sees zero-crash rows. It only learns from history where something actually happened, asking: given a crash occurred, what's the expected count?

Why Poisson loss? Crash counts are whole numbers ≥ 0. Poisson loss is mathematically designed for this — it treats the prediction as a rate (λ) and guarantees predictions are always non-negative. It also penalizes errors at low counts more heavily than at high counts, which reflects reality: being off by 1 when the true count is 1 is a much bigger deal than being off by 1 when the true count is 10.

Why separate from Stage 1? If you train one model on all data including 98.5% zeros, the Poisson model's optimal lazy strategy becomes "always predict ~0.015 (the base rate)." The zero rows overwhelm the gradient signal and the model never properly learns multi-crash behavior. Isolating Stage 2 removes that noise entirely.

Tail Weighting Inside Stage 2

Even within crash-only windows, single-crash days dominate. Without correction, Stage 2 optimizes heavily for 1-crash days and mostly ignores rare 5-crash pileups.

Tail weighting applies an extra multiplier to high-count rows during training:

```
count = 1  → weight multiplier = 1.0   (no boost)
count = 3  → weight multiplier ≈ 2.8
count = 5  → weight multiplier ≈ 4.6
count = 10 → weight multiplier ≈ 5.8
```

Formula: `weight = 1.0 + 2.0 × log(1 + count)` for counts above 2

The weights are then normalized so the total gradient scale stays stable, and capped at 50 so one extreme outlier event can't dominate the entire training run.

Combining Stage 1 + Stage 2

```
λ = P(crash)   ×   E[count | crash]
         ↑                ↑
      Stage 1          Stage 2
   (calibrated)
```

λ is the combined expected crash rate for that segment in the next 24 hours.

This is then converted to a true crash probability using the Poisson formula:

```
P(at least 1 crash) = 1 - e^(-λ)
```

λ	P(≥1 crash)
0.01	~1%
0.1	~9.5%
0.5	~39%
1.0	~63%

What the Routing Engine Receives

For every road segment, the engine gets a single number: **P(≥1 crash in next 24h)**. This becomes a penalty weight on that road edge. The pathfinding algorithm then balances travel time against crash risk — segments with high P(crash) are avoided unless the time cost of going around them is too high.

The full city scores in **~273ms**, fast enough to recalculate in near real-time.

What λ and P(≥1 crash) Mean

λ is a rate, not a probability

λ (lambda) answers: **"how many crashes would you expect here on average?"**

It can be any positive number — including fractions:

- λ = 0.1 → "on average, 1 crash every 10 days on this road"
- λ = 0.5 → "on average, 1 crash every 2 days"
- λ = 1.0 → "on average, 1 crash per day"
- λ = 2.0 → "on average, 2 crashes per day"

But λ alone doesn't answer the routing question. The routing engine doesn't care about averages — it cares about **"will a driver encounter a crash TODAY?"**

The conversion: rate → probability

That's what `P(≥1 crash) = 1 - e^(-λ)` does. It converts an average rate into the probability that at least one crash actually happens in this specific 24-hour window.

λ	Plain English	P(crash today)
0.01	Very quiet road, crash maybe once a y	~1%
0.1	Crash roughly once every 10 days	~9.5%
0.5	Crash roughly every other day	~39%
1.0	Crash roughly every day	~63%

Notice that even at λ=1.0 (one crash *on average* per day), the probability isn't 100% — because

some days might have 2 crashes and others have 0. The formula accounts for that randomness.

Why not just use λ directly?

Because λ is unbounded — it can be 0.1 or 5.0 or theoretically 100. Probabilities must sit between 0 and 1. The routing engine needs a clean 0–1 score to compare roads fairly and compute penalties. The formula is just the mathematically correct way to translate "expected rate" into "chance it happens at least once."